ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN

BÁO CÁO

MÔN: CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT NÂNG CAO

BÀI TẬP THỰC HÀNH

Ứng dụng từ điển tiếng Anh sử dụng Radix Trie

Sinh viên thực hiện:

Vũ Cao Mạnh Hùng 24520635

TP. Hồ Chí Minh, tháng 4 năm 2026

Mục lục

1 Tổng quan ứng dụng **3**

I Giới thiệu . . . 3

II Mục tiêu . . . 3

III Công nghệ sử dụng . . . 4

2 Cấu trúc dữ liệu Radix Trie **5**

I Định nghĩa . . . 5

II Cấu trúc node . . . 5

III Các thao tác chính . . . 5

III.1 Thêm từ – `insert(word, definition)` . . . 6

III.2 Tìm kiếm – `search(word)` . . . 6

III.3 Xóa từ – `delete(word)` . . . 6

IV Minh hoạ thay đổi cấu trúc cây . . . 7

IV.1 Trạng thái ban đầu (rỗng) . . . 7

IV.2 Sau `insert("apple", ...)` . . . 7

IV.3 Sau `insert("application", ...)` . . . 7

IV.4 Sau `insert("algorithm", ...)` . . . 7

IV.5 Sau `delete("apple")` . . . 7

V Độ phức tạp . . . 8

3 Giao diện ứng dụng **9**

I Bố cục tổng thể . . . 9

II Luồng xử lý thao tác . . . 9

III Dữ liệu mẫu . . . 9

4 Cài đặt và Hướng dẫn sử dụng **11**

I Yêu cầu hệ thống . . . 11

II Hướng dẫn cài đặt và chạy . . . 11

III Hướng dẫn sử dụng . . . 11

III.1 Thêm từ mới . . . 11

III.2 Tìm nghĩa từ . . . 12

III.3 Xóa từ . . . 12

IV Cấu trúc mã nguồn . . . 12

5 Kết luận **13**

I Kết quả đạt được . . . 13

II Hướng phát triển . . . 13

III Tài liệu tham khảo 13

Chương 1: Tổng quan ứng dụng

I. Giới thiệu

Ứng dụng **English Dictionary** là một từ điển tiếng Anh tích hợp giao diện đồ họa (GUI) được xây dựng bằng ngôn ngữ lập trình **Python** và thư viện **Tkinter**. Điểm cốt lõi của ứng dụng là cấu trúc dữ liệu **Radix Trie** (còn gọi là *Patricia Trie*) — một dạng cây nén tiền tố (compressed prefix tree) cho phép lưu trữ và tra cứu từ vựng với độ phức tạp $O(m)$, trong đó m là độ dài từ cần xử lý.
Ứng dụng hỗ trợ đầy đủ ba thao tác cơ bản trên từ điển:

- **Thêm** một mục từ mới kèm định nghĩa.
- **Tìm** nghĩa của một từ theo từ khóa.
- **Xóa** một mục từ khỏi từ điển.

Sau mỗi thao tác, cấu trúc cây Radix Trie được hiển thị trực quan dưới dạng ASCII, giúp người dùng quan sát rõ sự thay đổi của chỉ mục dữ liệu.

II. Mục tiêu

- Cài đặt cấu trúc dữ liệu Radix Trie từ đầu (không sử dụng thư viện bên ngoài).
- Xây dựng giao diện người dùng trực quan, dễ sử dụng bằng Tkinter.
- Trực quan hóa sự thay đổi của chỉ mục sau mỗi thao tác thêm, xóa, tìm kiếm.
- Đảm bảo hiệu năng tra cứu tốt hơn so với tìm kiếm tuyến tính thông thường.

III. Công nghệ sử dụng

Thành phần	Mô tả
Python 3.10+	Ngôn ngữ lập trình chính
Tkinter	Thư viện giao diện đồ họa (có sẵn trong Python)
Radix Trie	Cấu trúc dữ liệu chỉ mục, tự cài đặt
VS Code	Môi trường phát triển
GitHub	Lưu trữ và chia sẻ mã nguồn

Bảng 1.1: Công nghệ sử dụng trong dự án

Chương 2: Cấu trúc dữ liệu Radix Trie

I. Định nghĩa

Radix Trie (hay *Patricia Trie – Practical Algorithm To Retrieve Information Coded In Alphanumeric*) là biến thể nén của Trie tiêu chuẩn. Thay vì lưu từng ký tự riêng lẻ trên mỗi cạnh, Radix Trie **gộp các chuỗi con liên tiếp không phân nhánh** thành một nhãn cạnh duy nhất, từ đó giảm đáng kể số lượng node và bộ nhớ sử dụng.

> ***Trie tiêu chuẩn*** *lưu từng ký tự trên một cạnh riêng biệt.* ***Radix Trie*** *nén các chuỗi con liên tục vào một cạnh duy nhất, tiết kiệm bộ nhớ và tăng tốc độ duyệt.*

II. Cấu trúc node

Mỗi node trong Radix Trie bao gồm:

- `children`: từ điển ánh xạ ký tự đầu tiên của nhãn cạnh đến cặp `(node_con, nhãn_cạnh)`.
- `is_end`: cờ đánh dấu node là kết thúc của một từ hoàn chỉnh.
- `definition`: chuỗi định nghĩa của từ (chỉ có nghĩa khi `is_end = True`).
- `word`: từ hoàn chỉnh được lưu tại node kết thúc.

```python
class RadixTrieNode:
    def __init__(self):
        # key -> (child_node, edge_label)
        self.children: dict[str, tuple["RadixTrieNode", str]] = {}
        self.is_end: bool = False
        self.definition: str = ""
        self.word: str = ""
```

Listing 2.1: Lớp RadixTrieNode

III. Các thao tác chính

III.1. Thêm từ – `insert(word, definition)`

Thuật toán chèn duyệt từ root theo từng ký tự của `word`. Tại mỗi bước, tìm cạnh có nhãn bắt đầu bằng ký tự hiện tại và tính độ dài tiền tố chung (*common prefix*). Ba trường hợp xảy ra:

1. **Không tìm thấy cạnh phù hợp:** tạo node lá mới với nhãn bằng phần còn lại của từ.
2. **Nhãn cạnh được tiêu thụ hoàn toàn:** tiếp tục đệ quy xuống node con.
3. **Tiền tố chung ngắn hơn nhãn cạnh: tách cạnh** (*split*) tại điểm phân kỳ, tạo node trung gian mới.

```
# T ch  c nh   t i   v   tr  cp (common prefix length)
split_node = RadixTrieNode()
# C h u y n  node con c   x u ng  d   i  split_node
split_node.children[edge_label[cp]] = (child_node, edge_label[cp:])
#  T o  node l  cho  t   m i
new_leaf = RadixTrieNode()
new_leaf.is_end = True
new_leaf.definition = definition
new_leaf.word = word
split_node.children[remaining[cp]] = (new_leaf, remaining[cp:])
#  C p  n h t  c nh  cha  t r  v  split_node
node.children[first_char] = (split_node, edge_label[:cp])
```

Listing 2.2: Lõi thuật toán insert – xử lý split node

III.2. Tìm kiếm – `search(word)`

Duyệt từ root, theo từng cạnh khớp với phần còn lại của `word`. Nếu duyệt hết `word` và node hiện tại có `is_end = True`, trả về `definition`; ngược lại trả về `None`.

> **Độ phức tạp:** $T_{\text{search}} = O(m)$, với m là độ dài từ cần tìm. Không phụ thuộc vào kích thước từ điển n.

III.3. Xóa từ – `delete(word)`

Thuật toán xóa thực hiện đệ quy:

1. Duyệt đến node kết thúc của từ, đặt `is_end = False`.
2. Trong quá trình quay lui (*backtrack*), kiểm tra và **hợp nhất** (*merge*) các node trở nên không cần thiết (node không phải kết thúc và chỉ có đúng một node con).
3. Xóa hoàn toàn node lá nếu không còn con nào.

> *Việc hợp nhất node tự động đảm bảo cây luôn ở dạng nén tối ưu sau mỗi thao tác xóa, duy trì tính chất Radix Trie.*

IV. Minh hoạ thay đổi cấu trúc cây

Phần này minh họa cụ thể sự thay đổi của Radix Trie qua từng thao tác.

IV.1. Trạng thái ban đầu (rỗng)

```
ROOT
(chua co tu nao)
```

IV.2. Sau `insert("apple", ...)`

```
ROOT
          [apple]            apple
```

IV.3. Sau `insert("application", ...)`

Node `[apple]` được tách tại vị trí 4 (tiền tố chung `"app"`):

```
ROOT
          [app]
              [le]                 apple
              [lication]           application
```

IV.4. Sau `insert("algorithm", ...)`

```
ROOT
          [al]
                [gorithm]            algorithm
                [go...]       (cac tu bat dau bang "al" khac)
          [app]
              [le]                 apple
              [lication]           application
```

IV.5. Sau `delete("apple")`

Node `[le]` bị xóa. Node `[app]` chỉ còn một con nên được hợp nhất:

```
ROOT
          [algorithm]           algorithm
          [application]          application
    (node [app] da duoc merge vao [application])
```

V. Độ phức tạp

Thao tác	Thời gian	Không gian bổ sung
`insert(word, def)`	$O(m)$	$O(m)$
`search(word)`	$O(m)$	$O(1)$
`delete(word)`	$O(m)$	$O(1)$
`get_all_words()`	$O(n \cdot m)$	$O(n \cdot m)$
`get_trie_structure()`	$O(n \cdot m)$	$O(n \cdot m)$

Bảng 2.1: Độ phức tạp các thao tác (m = độ dài từ, n = số từ trong từ điển)

Không gian lưu trữ tổng thể: $O(n \cdot m)$ trong trường hợp xấu nhất (không có từ nào chia sẻ tiền tố). Trong thực tế, Radix Trie tiết kiệm bộ nhớ đáng kể nhờ cơ chế nén tiền tố chung.

Chương 3: Giao diện ứng dụng

I. Bố cục tổng thể

Giao diện được xây dựng bằng **Tkinter** với bố cục chia làm hai vùng chính thông qua `PanedWindow` có thể kéo để điều chỉnh kích thước:

Vùng giao diện	Thành phần & Chức năng
Panel trái	Card thao tác (Thêm / Tìm / Xóa từ) và danh sách toàn bộ từ (`Listbox`) hiển thị theo thứ tự bảng chữ cái.
Tab Result	Hiển thị kết quả thao tác với màu phân biệt: xanh lá = thành công, đỏ = lỗi, xanh dương = thông tin.
Tab Trie Structure	Cây Radix Trie dạng ASCII tự động làm mới sau mỗi thao tác, hiển thị ký hiệu tại node kết thúc.
Tab Operation Log	Nhật ký thao tác kèm timestamp `[HH:MM:SS]`, hỗ trợ xóa log.

Bảng 3.1: Bố cục giao diện ứng dụng

II. Luồng xử lý thao tác

1. Người dùng nhập thông tin vào ô `Entry` trên Panel trái.
2. Nhấn nút tương ứng (Thêm / Tìm / Xóa).
3. Ứng dụng gọi phương thức tương ứng trên đối tượng `RadixTrie`.
4. Kết quả trả về được hiển thị tại Tab **Result**.
5. Danh sách từ (`Listbox`) và cây Trie (**Trie Structure**) được làm mới.
6. Thao tác được ghi vào Tab **Operation Log**.

III. Dữ liệu mẫu

Ứng dụng khởi động với **28 từ mẫu** được nạp sẵn, bao phủ nhiều tiền tố chung (ví dụ: *apple / application / algorithm*, *stack / string*, *queue / quick*) nhằm minh họa rõ cơ chế

nén của Radix Trie ngay khi mở ứng dụng.

Chương 4: Cài đặt và Hướng dẫn sử dụng

I. Yêu cầu hệ thống

- Python **3.10** trở lên.
- Hệ điều hành: Windows, macOS hoặc Linux.
- Không cần cài thêm thư viện ngoài — Tkinter có sẵn trong bản cài Python tiêu chuẩn.

II. Hướng dẫn cài đặt và chạy

Hệ thống yêu cầu cài đặt sẵn Python 3.7 trở lên và thư viện Tkinter (đã được tích hợp sẵn trong Python). Để khởi chạy ứng dụng, thực thi các lệnh sau trong Terminal:

```
# Clone repository
git clone https://github.com/Waterbear566/Radix-Trie-Dictionary
cd Radix-Trie-Dictionary

# Chay ung dung
python dictionary.py
```

Ngoài ra, người dùng có thể truy cập mã nguồn trực tiếp tại hệ thống quản lý phiên bản GitHub:

- **Địa chỉ Repository:** `https://github.com/Waterbear566/Radix-Trie-Dictionary` Source Code trên GitHub

> *Bạn có thể tải toàn bộ mã nguồn dưới dạng tệp nén (.zip) bằng cách truy cập liên kết trên, chọn nút **Code** và nhấn **Download ZIP**.*

III. Hướng dẫn sử dụng

III.1. Thêm từ mới

1. Nhập từ vào ô **Word** trong phần *Add Word*.

2. Nhập định nghĩa vào ô **Definition**.
3. Nhấn nút **Add Word**.
4. Quan sát: danh sách từ và cây Trie cập nhật ngay lập tức.

III.2. Tìm nghĩa từ

1. Nhập từ cần tra vào ô **Word** trong phần *Search Word*.
2. Nhấn nút **Search**.
3. Kết quả hiển thị tại Tab **Result**; từ tương ứng được tô sáng trong danh sách.

III.3. Xóa từ

1. Nhập từ cần xóa vào ô **Word** trong phần *Delete Word*.
2. Nhấn nút **Delete Word**.
3. Quan sát: cây Trie tự động hợp nhất node nếu cần thiết.

IV. Cấu trúc mã nguồn

Listing 4.1: Cấu trúc thư mục dự án

```
english-dictionary-radix-trie/
          dictionary.py        # Toan bo logic: RadixTrie + GUI
                RadixTrieNode    # Class dinh nghia node
                RadixTrie        # insert / search / delete / visualize
                DictionaryApp    # Giao dien Tkinter (ke thua tk.Tk)
          README.md            # Huong dan su dung
```

Chương 5: Kết luận

I. Kết quả đạt được

- Cài đặt thành công cấu trúc dữ liệu **Radix Trie** từ đầu bằng Python, đầy đủ ba thao tác: `insert`, `search`, `delete` với độ phức tạp $O(m)$.
- Xây dựng giao diện đồ họa trực quan bằng **Tkinter** chạy ngay trên VS Code mà không cần cài thêm thư viện nào.
- Trực quan hóa cây Radix Trie dạng ASCII, thể hiện rõ cơ chế *split node* khi thêm từ và *merge node* khi xóa từ.
- Hệ thống nhật ký thao tác giúp theo dõi toàn bộ lịch sử hoạt động.

II. Hướng phát triển

- Thêm chức năng **tự động gợi ý từ** (autocomplete) dựa trên tiền tố nhập vào.
- Hỗ trợ **lưu và tải từ điển** từ file JSON/CSV.
- Thêm chức năng **tìm kiếm mờ** (fuzzy search) cho phép tra cứu gần đúng.
- Mở rộng sang từ điển **đa ngôn ngữ** (Anh – Việt, Anh – Pháp, ...).
- Triển khai phiên bản **web** sử dụng Flask hoặc FastAPI.

III. Tài liệu tham khảo

1. Morrison, D. R. (1968). *PATRICIA — Practical Algorithm to Retrieve Information Coded in Alphanumeric*. Journal of the ACM, 15(4), 514–534.
2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
3. Python Software Foundation. *tkinter — Python interface to Tcl/Tk*. `https://docs.python.org/3/library/tkinter.html`
4. Wikipedia. *Radix tree*. `https://en.wikipedia.org/wiki/Radix_tree`